

Step 4 — Production Wrap — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way (Project 5's Step 1 pattern, pointed at CodeGuard)

```
# api/schemas.py
from pydantic import BaseModel

class ReviewRequest(BaseModel):
    diff: str

class ReviewResponse(BaseModel):
    compiled_review: str

# api/routes.py
from fastapi import APIRouter
from supervisor import run_codeguard
from api.schemas import ReviewRequest, ReviewResponse

router = APIRouter()

@router.post("/review", response_model=ReviewResponse)
def review(request: ReviewRequest):
    result = run_codeguard(request.diff)
    return ReviewResponse(compiled_review=result)

@router.get("/health")
def health():
    return {"status": "ok"}
```

This is exactly Project 5 Step 1's Basic Version, with `topic` swapped for `diff` and `final_report` swapped for `compiled_review`. It works, and it has the exact same gaps Project 5's own Basic Version has — no error handling, and a blocking call directly inside a synchronous route.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the full Project 5 stack, reused: FastAPI + SQLite + Dockerfile

```
# api/routes.py
import logging
from uuid import uuid4
from datetime import datetime, timezone
from fastapi import APIRouter, HTTPException
from starlette.concurrency import run_in_threadpool
from db.database import SessionLocal
from db.models import Run
from supervisor import run_codeguard
from api.schemas import ReviewRequest, ReviewResponse

logger = logging.getLogger("codeguard.api")
router = APIRouter()

@router.post("/review", response_model=ReviewResponse)
async def review(request: ReviewRequest) -> ReviewResponse:
    run_id = str(uuid4())
    logger.info("review %s starting", run_id)

    with SessionLocal() as session:
        session.add(Run(id=run_id, input=request.diff, status="pending", created_at=datetime.now(timezone.utc)))
        session.commit()

    try:
        result = await run_in_threadpool(run_codeguard, request.diff)
    except Exception:
        logger.exception("review %s failed", run_id)
        raise HTTPException(status_code=500, detail="Review failed – please try again")

    with SessionLocal() as session:
        run = session.get(Run, run_id)
        run.output = result
        run.status = "success"
        run.completed_at = datetime.now(timezone.utc)
        session.commit()
```

```

        return ReviewResponse(compiled_review=result)

@router.get("/runs/{run_id}")
def get_run(run_id: str) -> Run:
    with SessionLocal() as session:
        run = session.get(Run, run_id)
    if not run:
        raise HTTPException(status_code=404, detail="Run not found")
    return run

```

```

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Difference from Basic: every piece here — `run_in_threadpool`, the safe error handling, the `Run` table, the two-phase `"pending" → "success"` write, the layer-cached Dockerfile — is Project 5's own Intermediate/Advanced solution, reused directly, exactly as the README instructs ("no new ideas here — just repetition"). The only genuinely new content is the request/response shape (`diff` in, `compiled_review` out) and which function gets called (`run_codeguard` instead of `graph.invoke`).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — an eval suite that specifically tests routing, not just reviewer accuracy

```
# eval_suite/tasks.py
EVAL_TASKS = [
    {
        "name": "planted_secret_flagged_critical",
        "diff": 'diff --git a/config.py b/config.py\n+ API_KEY = "sk-realistic-lo',
        "check": lambda review: "critical" in review.lower(),
    },
    {
        "name": "docs_only_diff_skips_security",
        "diff": 'diff --git a/README.md b/README.md\n+ Some documentation text.',
        "check": lambda review, applicable_reviewers: "security" not in applicabl
    },
    {
        "name": "source_change_no_test_flagged",
        "diff": 'diff --git a/app.py b/app.py\n+ def new_feature(): pass',
        "check": lambda review: "test" in review.lower(),
    },
]
```

This directly tests Step 3's routing logic (`docs_only_diff_skips_security`), not just each reviewer answering correctly in isolation — the gap Hint 2's Intermediate level pointed at. A suite that only tested reviewer accuracy could pass even with Step 3's routing completely broken (every reviewer running on every diff), since over-running a reviewer doesn't make its individual answers wrong — it just wastes money, which this suite alone wouldn't catch without a task designed specifically to check it.

Approach 2 — the style guide as a mounted volume, not baked into the image

```
# Dockerfile — style_guide.md is NOT copied in; it's mounted at run time
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

```
# style_guide.md is intentionally excluded via .dockerignore
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# run time: mount the real style guide in, rebuild the vectorstore from it once a
docker run --env-file .env -v $(pwd)/style_guide.md:/app/style_guide.md -p 8000:8
```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's eval suite proves each reviewer works but says nothing about whether Step 3's routing is actually saving any money — a completely broken router (running every reviewer on every diff) would still pass every task in Intermediate's suite. Approach 1 closes that gap with a task built specifically to test routing. Approach 2 is a separate, genuinely new concern Project 5 never had: CodeGuard's Style reviewer depends on a real document that isn't a secret but *does* change independently of the code — baking it into the image (Project 5's default pattern for everything else) means rebuilding the whole image every time the style guide changes, where mounting it as a volume lets the guide update without a rebuild.

Which one should you actually write? Approach 1's routing-aware eval task, without question — it's the one part of this step that's genuinely CodeGuard-specific rather than a copy of Project 5, and skipping it means your test suite could pass while Step 3's actual routing logic is silently broken. Approach 2's volume-mount is worth doing once you notice the style guide will realistically change more often than your application code does — for a first pass, baking it in (Intermediate's approach) is a perfectly fine simplification, same as skipping this whole optional step is fine for a first pass at the project overall.